

모듈을 배우다 버그를 찾다

커널 모듈 동작 원리 실습에서 세 번째 패치까지

앞의 두 패치는 문서의 낡은 문장과 쓰이지 않는 선언을 지우는 정리였다. 이번 문서는 성격이 다르다. 리눅스 커널이 모듈을 어떻게 올리고 내리는지 공부하려고 가상 머신에서 이것저것 해 보다가, 재현되는 커널 경고를 발견하고 그것을 고쳐 보내기까지의 기록이다. 소재를 찾으러 간 것이 아니라 배우다가 걸려 넘어진 경우이고, 그래서 과정의 대부분은 “왜 이렇게 동작하지”를 확인하는 일이다.

2026년 9월 16--17일 작업 기록. 보낸 패치: media: vivid: initialize control update work items early

1. 출발점

1.1 주제와 지형

주제는 모듈 동작 원리로 잡았다. 먼저 소스가 어디에 얼마나 있는지부터 봤다. 모듈 코어는 kernel/module/에 모여 있고 .c 파일 17개, 그중 main.c가 4001줄이다. 읽을 순서를 잡기 위해 함수 위치를 미리 표시해 두었다.

```
include/linux/module.h:306  enum module_state (LIVE/COMING/GOING/UNFORMED)
include/linux/module.h:397  struct module
kernel/module/main.c:3812   finit_module      (syscall entry)
                           :3433   load_module      (main path)
                           :2954   layout_and_allocate
                           :1529   simplify_symbols / :391 find_symbol
                           :3318   complete_formation (state -> COMING)
                           :3088   do_init_module      (calls module_init)
                           :804    delete_module / :1417 free_module
```

한 가지 미리 확인해 둘 것이 있었다. 모듈 서브시스템은 지금까지 패치를 보내 온 media와 트리도 리스트도 다르다. MAINTAINERS의 MODULE SUPPORT 항목은 기존 트리를 modules-next로, 리스트를 linux-modules@vger.kernel.org로 적고 있다. 여기에 패치를 보내려면 트리를 따로 받아야 한다는 뜻이다.

1.2 활동량 차이

기여할 곳을 가늠하려고 최근 1년 커밋 수를 세어 봤다. 코어(kernel/module/)는 36개인데, 모듈 셀프테스트 세 곳(tools/testing/selftests/module, kmod, lib/tests/module)은 0개였다. selftests/module에는 스크립트가 하나뿐이다. 코어는 계속 손보는데 시험은 따라가지 못하는 상태로 보였다.

다만 이것은 “활동량이 적다”는 관찰이지 “여기 버그가 있다”는 근거가 아니다. 그래서 먼저 직접 먼저 보기로 했다. 만들어 두고 한 번도 부팅하지 않았던 가상 머신을 띄우는 것이 첫 일이 됐다.

2. 관찰

2.1 모듈 하나를 올리면 생기는 일

가상 머신은 모듈을 하나도 올리지 않은 상태로 시작한다. 실습에는 그 편이 낫다. 지금 올라와 있는 것은 전부 내가 올린 것이기 때문이다.

```
# modprobe vivid
# lsmod | head -8
Module                Size  Used by
vivid                 688128  0
videobuf2_vmalloc     24576  1 vivid
videobuf2_dma_contig  36864  1 vivid
videobuf2_memops      12288  2 videobuf2_vmalloc,videobuf2_dma_contig
v4l2_tpg              151552  1 vivid
cec                   143360  1 vivid
videobuf2_v4l2        73728  1 vivid
```

명령 하나에 일곱 개가 넘게 올라왔다. vivid는 혼자 돌지 못하고 버퍼 관리와 테스트 패턴 생성기, HDMI 제어 같은 공통 코드를 다른 모듈에서 빌려 쓴다. modprobe가 의존 관계를 보고 아래쪽부터 차례로 올린 것이다.

세 열의 출처는 /proc/modules다. kernel/module/procfs.c:86이 이름과 크기를 찍고, 바로 아래 print_unload_info()가 참조 수와 붙잡고 있는 모듈 이름을 찍는다. 크기는 module_total_size()가 mod->mem[]의 모든 종류를 더한 값이고, 값이 전부 4096의 배수인 것은 페이지 단위로 할당하기 때문이다. videobuf2_memops의 "2"처럼 홀더가 둘이면 그 모듈을 먼저 내릴 수 없다.

2.2 참조 수를 직접 움직여 보기

/sys/module/vivid/에는 커널이 모듈마다 만들어 주는 파일들이 있다. 그중 refcnt와 initstate를 보면서 장치를 열고 닫아 봤다.

```
# cat /sys/module/vivid/refcnt /sys/module/vivid/initstate
0
live
# exec 3</dev/video0          (keep the device open)
# cat /sys/module/vivid/refcnt
1
# rmmod vivid; echo "rc=$?"
rmmod: ERROR: Module vivid is in use
rc=1
# exec 3<&-                    (close it)
# cat /sys/module/vivid/refcnt
0
# rmmod vivid; echo "rc=$?"
rc=0
```

여기서 궁금한 것은 "누가 참조를 올렸는가"였다. 답은 vivid도 V4L2 공통 코드도 아니었다. 장치 노드를 여는 순간 문자 장치 계층이 올라간다. fs/char_dev.c:350의 chrdev_open()이 try_module_get(owner)를 부르고, 닫을 때 같은 파일 :363에서 module_put()을 부른다. v4l2_open()이 하는 video_get()은 장치 구조체 참조이지 모듈 참조가 아니다.

그 owner가 왜 vivid인지도 따라가 봤다. vivid가 video_register_device()를 부르면 그 인라인 함수가 vdev->fops->owner를 넘기고(include/media/v4l2-dev.h:386), vivid의 파일 연산 구조체는 .owner = THIS_MODULE이다. 그 값이 v4l2-dev.c:1061에서 vdev->cdev->owner로 들어간다. 즉 장

치를 열면 그 노드를 등록한 드라이버 모듈이 붙잡힌다. 공통 코드가 대신 잡아 주는 구조였다면 사용 중에도 vivid를 내릴 수 있었을 것이다.

rmmod가 막힌 지점은 `delete_module(main.c:804)`에서 `try_stop_module(:772)`로 들어가, 참조를 놓지 못하면 강제 플래그가 없는 한 `-EWOULDBLOCK`을 돌려주는 곳이다. 성공하면 그 자리에서 상태가 `MODULE_STATE_GOING`으로 바뀐다. 앞에서 파일로 읽은 `initstate`가 여기서 움직인다.

2.3 파일 목록이 알려 주는 설정

`initssize`가 0인 것도 이유가 있다. `do_init_module(main.c:3088)`이 초기화를 마친 뒤 `init` 전용 메모리를 반납하기 때문에, 살아 있는 모듈에서 읽으면 늘 0이다. 그래서 `coresize`와 `lsmod`의 `Size`가 같아진다.

목록에 `datasize`가 없다는 점도 코드와 맞는다. 그 속성은 아키텍처가 모듈 데이터를 따로 잡는 설정일 때만 만들어진다(`main.c:1044`). 파일 목록만 보고도 이 커널의 설정을 읽어 낼 수 있다는 뜻이다.

3. 실패를 만들어 보기

3.1 첫 시도는 실패를 만들지 못했다

성공 경로만 봤으니 이번에는 적재가 실패하는 경로를 보고 싶었다. 인스턴스를 과하게 요구해 보았다.

```
# modprobe vivid n_devs=99
# dmesg | tail -1
videodev: could not get a free minor
# cat /sys/module/vivid/parameters/n_devs
25
```

모듈은 그대로 올라왔다. 코드를 보니 의도된 동작이었다. 요청한 99는 먼저 `clamp_t`로 64까지 잘리고 (`CONFIG_VIDEO_VIVID_MAX_DEVS`), 인스턴스를 만들다가 V4L2 코어의 마이너 번호가 바닥나 26번째에서 실패했다. 그런데 루프에 이런 줄이 있다.

```
ret = vivid_create_instance(pdev, i);
if (ret) {
    /* If some instantiations succeeded, keep driver */
    if (i)
        ret = 0;
    break;
}
```

하나라도 성공했으면 오류를 지운다. 바로 다음 줄에서 `n_devs = i`;로 실제 생성 수를 인자 파일에 반영한다. 숫자는 정확히 맞아떨어졌다. 인스턴스 하나가 노드 10개를 쓰고 25개를 만들었으니 250개, 남은 6개로는 26번째를 채울 수 없다. `/dev/video*`가 정확히 100개인 것은, 부분 생성된 26번째 인스턴스의 노드가 되돌려졌다는 뜻이기도 하다.

실패 유도에는 실패했지만 대신 설계를 하나 배웠다. 그리고 “요청 99, 실제 25”라는 사실을 사용자에게 알려 주는 메시지가 없다는 것도 눈에 남았다.

3.2 두 번째 시도

이번에는 모순된 인자를 줬다. `node_types`는 어떤 노드를 만들지 고르는 비트마스크다. 입력은 있는데 받을 노드를 모두 끄면 드라이버가 스스로 거부한다.

```
# modprobe vivid node_types=0xc1d3c; echo "rc=$?"
rc=0
# dmesg | tail -3
---[ end trace 0000000000000000 ]---
vivid: error -22 while loading driver
vivid vivid.0: probe with driver vivid failed with error -22
```

여기서 두 가지를 알게 됐다. 하나는 **probe 실패와 모듈 적재 실패가 다르다**는 것이다. vivid는 platform 드라이버라서 module_init은 드라이버 등록까지만 하고 장치는 그 뒤 probe에서 만든다. probe가 실패해도 모듈은 남는다. 그래서 modprobe의 종료 코드가 0이고 /sys/module/vivid도 그대로 있다.

다른 하나가 중요했다. 오류 메시지 바로 위에 end trace가 있었다. 커널이 경고를 찍었다는 뜻이다.

4. 발견

4.1 경고의 정체

전체 로그를 파일로 떼서 읽었다. 같은 경고가 두 번 나 있었다.

```
WARNING: kernel/workqueue.c:4400 at __flush_work+0x9a7/0xc20, CPU#2: modprobe
cancel_work_sync+0x90/0xc0
vivid_dev_release+0x1e/0x170 [vivid]
v4l2_device_put+0x80/0xb0
vivid_probe+0x257c/0xa3e0 [vivid]
...
do_init_module+0x280/0x860
__x64_sys_finit_module+0xc3/0x130
```

workqueue.c:4400은 if (WARN_ON(!work->func))이다. 즉 한 번도 INIT_WORK 되지 않은 work를 취소하려 했다는 뜻이다.

경로를 코드에서 맞춰 보면 이렇다. 인스턴스 생성이 실패하면 정리 라벨로 건너뛰고, 거기서 v4l2_device_put()을 부른다. 마지막 참조가 떨어지면 release 콜백인 vivid_dev_release()가 불리고, 그 함수의 첫 두 줄이 두 work를 취소한다. 그런데 그 work를 초기화하는 INIT_WORK 두 줄은 훨씬 뒤에 있었고, 실패로 빠지는 goto 여섯 곳은 전부 그보다 앞이었다. 구조체는 kzalloc으로 잡으므로 work->func은 NULL인 상태다.

4.2 수정 범위를 좁히기

release 콜백은 work 말고도 여러 가지를 정리한다. 그것들도 초기화 전 상태를 건디는지 하나씩 봤다.

- 컨트롤 핸들러 해제는 if (!hdl)과 if (!hdl->buckets) 검사가 있어 안전하다.
- 테스트 패턴 생성기 해제는 전부 vfree(NULL) 패턴이라 안전하다.
- 미디어 장치 정리는 초기화가 실패 지점보다 앞이라 안전하다.

결국 초기화 전 상태를 건디지 못하는 것은 cancel_work_sync 두 줄뿐이었다. 고칠 범위가 좁다는 것이 확인됐다.

4.3 언제부터 있었나

git log -S로 두 work가 들어온 커밋을 찾으니까 2024년 6월의 d7c969f37515 "media: vivid: Add 'Is Connected To' menu controls"였다. work 두 개와 release에서 하나를 취소하는 코드가 같이 들어왔다.

그리고 2026년 9월, 다른 사람이 나머지 하나도 취소하도록 고친 커밋이 있었다(1adaefca7111). 그 패치는 work가 kfree(dev) 뒤에 도는 use-after-free를 막은 것으로, Fixes:와 Cc: stable을 달고 “사내 정적 분석 도구로 찾았다”고 적혀 있었다. 같은 드라이버에서 비슷한 성격의 수정이 또 있었다(a07c179a92e9, 실패 경로의 참조 누수).

두 선례가 알려 주는 것은 분명했다. 정리 경로의 버그는 이 드라이버에서 지금도 계속 발견되고 받아들여진다. 다만 그 둘은 도구가 찾은 것이고, 이번 것은 실행해서 재현한 것이다.

5. 중복 확인

보내기 전에 같은 보고가 있었는지 확인했다. 파일 이름으로 한 번, 증상으로 한 번, 함수 이름으로 한 번.

```
lore.kernel.org/linux-media/tq=dfn%3Adrivers%2Fmedia%2Ftest-drivers%2Fvivid%2Fvivid-core.c
linux-media.vger.kernel.org archive mirror
dfn:drivers/media/te|search help / color / mirror / Atom feed
Search results ordered by [date] [relevance] view[summary] [nested] [Atom feed]
download mbox.gz: [results only] [full threads]

1. [PATCH] media: vivid: cancel svid ctrl work on release
   - by Fan Wu @ 2026-09-09 8:13 UTC [70%]
2. [PATCH] media: vivid: fix cleanup bugs in vivid_init()
   - by Guangshuo Li @ 2026-04-15 16:20 UTC [70%]
3. [PATCH v2 2/3] media: vivid: Optimize the code using vmalloc_array
   - by tanze @ 2025-10-22 3:20 UTC [72%]
4. [PATCH 2/3] media: vivid: Optimize the code using vmalloc_array
   - by tanze @ 2025-10-21 14:31 UTC [72%]
5. [PATCH v2 06/10] media: Replace void * with video_device_state * in all driver ioctl implementations
   - by Jai Luthra @ 2025-09-19 9:55 UTC [37%]
6. [PATCH 3/3] media: vivid: use vmalloc_array and vcalloc to simplify code
   - by Qianfeng Rong @ 2025-08-12 3:53 UTC [72%]
7. [PATCH v3 71/76] media: test-drivers: Rename second ioctl handlers argument to 'void *priv'
   - by Laurent Pinchart @ 2025-08-10 1:30 UTC [83%]
8. [PATCH v3 09/76] media: Replace file->private_data access with file_to_v4l2_fh()
   - by Laurent Pinchart @ 2025-08-10 1:29 UTC [77%]
9. [PATCH v2 71/76] media: test-drivers: Rename second ioctl handlers argument to 'void *priv'
   - by Laurent Pinchart @ 2025-08-09 21:16 UTC [83%]
10. [PATCH v2 09/76] media: Replace file->private_data access with file_to_v4l2_fh()
    - by Laurent Pinchart @ 2025-08-09 21:15 UTC [77%]
11. [PATCH 09/05] media: Replace file->private_data access with file_to_v4l2_fh()
    - by Jacopo Mondì @ 2025-08-02 9:22 UTC [69%]
```

파일 이름 검색. 맨 위 두 건이 앞에서 본 두 선례이고, 초기화 순서를 건드리는 나머지 패치는 없었다.

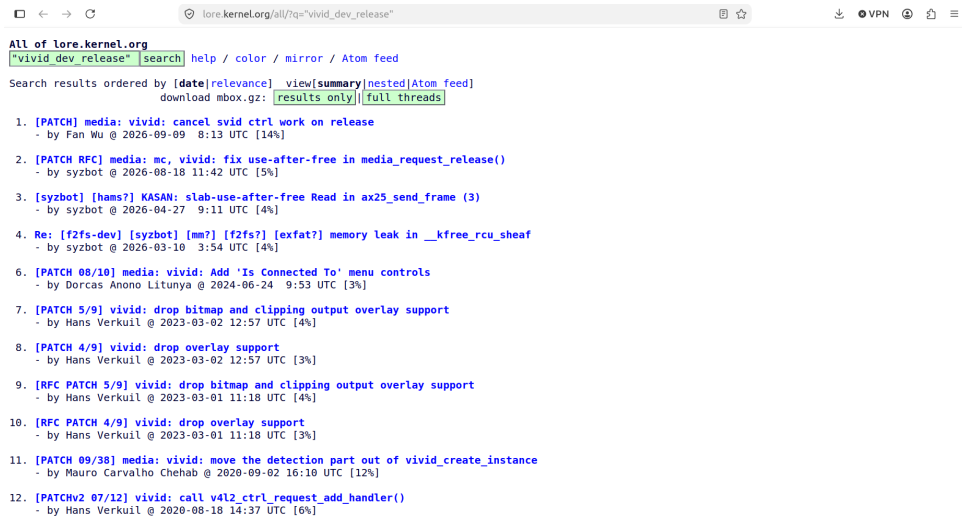
```
lore.kernel.org/all/tq=vivid_dev_release+cancel_work_sync
All of lore.kernel.org
vivid_dev_release ca|search help / color / mirror / Atom feed
Search results ordered by [date] [relevance] view[summary] [nested] [Atom feed]
download mbox.gz: [results only] [full threads]

1. [PATCH] media: vivid: cancel svid ctrl work on release
   - by Fan Wu @ 2026-09-09 8:13 UTC [17%]
2. [PATCH 08/10] media: vivid: Add 'Is Connected To' menu controls
   - by Dorcas Anono Litunya @ 2024-06-24 9:53 UTC [3%]

Results 1-2 of 2 | reverse | options above

This is an external index of several public inboxes,
see mirroring instructions on how to clone and mirror
all data and code used by this external index.
```

증상으로 검색한 결과는 두 건뿐이었다. 이 경고를 보고한 메일은 없다.



함수 이름 검색, syzbot 보고 중 vivid 관련은 다른 문제였다.

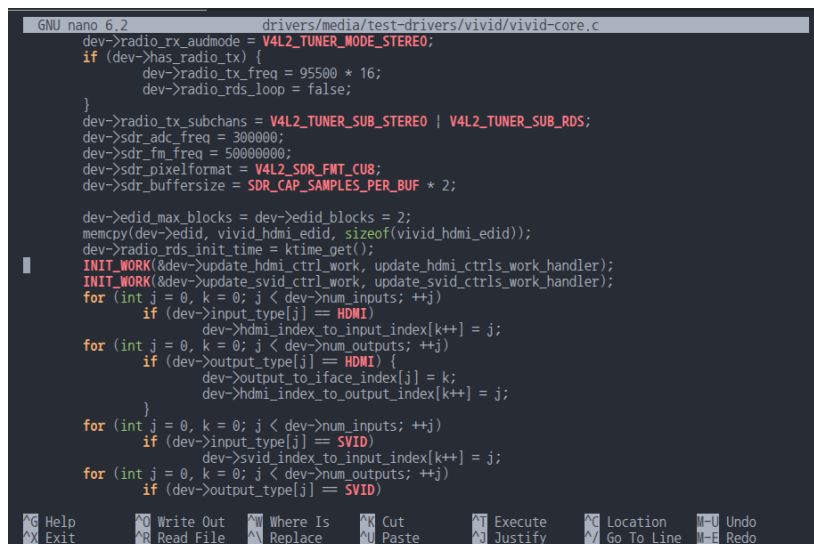
세 검색 모두 같은 결론이었다. 아직 보고되지 않은 문제로 판단했다.

6. 수정

6.1 어디에 넣을 것인가

고치는 방법은 여럿이었다. 초기화를 앞으로 옮기거나, release에서 조건부로만 취소하거나, 실패 경로가 release를 타지 않게 구조를 바꾸거나.

기준은 하나로 정리됐다. release 콜백이 등록되는 순간부터 그 함수는 불릴 수 있다. 그러므로 work 초기화는 그 줄보다 먼저 끝나 있어야 한다. 그렇다면 구조체를 할당한 직후에 초기화하는 것이 가장 단순하다. 그 사이 어디에 새 실패 경로가 추가돼도 안전하기 때문이다. INIT_WORK은 함수 포인터만 설정하므로 워크 큐나 다른 필드에 의존하지 않고, work 처리 함수도 전역 변수만 본다는 것을 미리 확인했다.



원래 자리. EDID와 라디오 기본값을 채우는 블록 한가운데에 있었다.

```

GNU nano 6.2 drivers/media/test-drivers/vivid/vivid-core.c *
int ccs_out = ccs_out_mode[inst];
bool has_tuner;
bool has_modulator;
struct vivid_dev *dev;
unsigned node_type = node_types[inst];
v4l2_std_id tvnorms_cap = 0, tvnorms_out = 0;
int ret;
int i;

/* allocate main vivid state structure */
dev = kzalloc_obj(*dev);
if (!dev)
    return -ENOMEM;

dev->inst = inst;

#ifdef CONFIG_MEDIA_CONTROLLER
dev->v4l2_dev.mdev = &dev->mdev;

/* Initialize media device */
strcpy(dev->mdev.model, VIVID_MODULE_NAME, sizeof(dev->mdev.model));
snprintf(dev->mdev.bus_info, sizeof(dev->mdev.bus_info),
    "platform:%s-%03d", VIVID_MODULE_NAME, inst);
dev->mdev.dev = &pdev->dev;
media_device_init(&dev->mdev);
dev->mdev.ops = &vivid_media_ops;
#endif

/* register v4l2 device */

```

웁길 자리. 구조체를 할당하고 인스턴스 번호를 넣은 직후다.

```

GNU nano 6.2 drivers/media/test-drivers/vivid/vivid-core.c *
int ccs_out = ccs_out_mode[inst];
bool has_tuner;
bool has_modulator;
struct vivid_dev *dev;
unsigned node_type = node_types[inst];
v4l2_std_id tvnorms_cap = 0, tvnorms_out = 0;
int ret;
int i;

/* allocate main vivid state structure */
dev = kzalloc_obj(*dev);
if (!dev)
    return -ENOMEM;

dev->inst = inst;
INIT_WORK(&dev->update_hdmi_ctrl_work, update_hdmi_ctrls_work_handler);
INIT_WORK(&dev->update_svid_ctrl_work, update_svid_ctrls_work_handler);

#ifdef CONFIG_MEDIA_CONTROLLER
dev->v4l2_dev.mdev = &dev->mdev;

/* Initialize media device */
strcpy(dev->mdev.model, VIVID_MODULE_NAME, sizeof(dev->mdev.model));
snprintf(dev->mdev.bus_info, sizeof(dev->mdev.bus_info),
    "platform:%s-%03d", VIVID_MODULE_NAME, inst);
dev->mdev.dev = &pdev->dev;
media_device_init(&dev->mdev);
dev->mdev.ops = &vivid_media_ops;
#endif

```

웁긴 뒤. 두 줄이 할당 직후로 올라왔다.

6.2 변경 내용

```

drivers/media/test-drivers/vivid/vivid-core.c | 4 +---
1 file changed, 2 insertions(+), 2 deletions(-)

```

```

dev->inst = inst;
+ INIT_WORK(&dev->update_hdmi_ctrl_work, update_hdmi_ctrls_work_handler);
+ INIT_WORK(&dev->update_svid_ctrl_work, update_svid_ctrls_work_handler);
...
dev->radio_rds_init_time = ktime_get();
- INIT_WORK(&dev->update_hdmi_ctrl_work, update_hdmi_ctrls_work_handler);
- INIT_WORK(&dev->update_svid_ctrl_work, update_svid_ctrls_work_handler);

```

7. 검증에서 두 번 헤맨 곳

7.1 디렉터리 빌드는 모듈 파일을 만들지 않는다

수정하고 경고 없이 빌드한 뒤 가상 머신에서 다시 재현했다. 경고가 그대로 두 개 나왔다. 코드가 틀린 줄 알고 한참 들여다봤지만 원인은 빌드 산출물이었다.

```
vivid-core.c 23:53:10 (edited)
vivid-core.o 23:53:23 (recompiled)
vivid.ko      21:47:46 (older than the object!)
```

디렉터리를 지정한 make는 그 디렉터리의 오브젝트까지만 만든다. 최종 모듈 파일은 그다음 단계에서 나오므로 make modules를 돌려야 갱신된다. 가상 머신은 모듈 파일을 올리니, 이 단계를 빼먹으면 옛 코드로 시험하고 “안 고쳐졌다”고 오판하게 된다.

경고를 확인하는 빌드와 실행으로 검증하는 빌드는 명령이 다르다. 검증 전에는 모듈 파일의 시각이 소스보다 새것인지 먼저 본다.

7.2 그냥 다시 만들면 커널이 거부한다

모듈 파일을 다시 만들고 올렸더니 이번에는 적재 자체가 거부됐다.

```
modprobe: ERROR: could not insert 'vivid': Exec format error
videobuf2_common: version magic '7.3.0-rc1+ SMP preempt mod_unload '
                  should be '7.3.0-rc1-virtme SMP preempt mod_unload '
```

실행 중인 커널은 가상 머신 도구가 빌드한 것이라 이름 끝에 -virtme가 붙는다. 그런데 트리에서 그냥 make modules를 돌리면 다른 문자열이 박힌다. 설정 파일이 아니라 빌드할 때 넘기는 인자로 붙는 값이라, 트리만 봐서는 알 수 없었다. 도구의 소스에서 그 줄을 찾아 같은 값을 주고 다시 링크하니 해결됐다.

이 재빌드는 vivid 하나가 아니라 모든 모듈을 바꿔 놓는다. 그래서 의존 모듈까지 한꺼번에 거부당했다. 교훈은 하나다. 검증용 빌드는 가상 머신 커널을 만든 것과 같은 방법으로 해야 한다.

8. 검증과 발송

8.1 숫자 두 개

```
$ ~/linux-media/vm.sh 'modprobe vivid node_types=0xc1d3c; ...'
trace=0
[ 24.532889] vivid: error -22 while loading driver
[ 24.532903] vivid vivid.0: probe with driver vivid failed with error -22
```

수정 전에는 적재 한 번에 경고가 두 개였다. 수정 후에는 0이다. 그리고 원래의 거부 처리는 그대로다. 이 두 가지가 패치의 근거 전부이고, 커밋 메시지에도 그대로 들어갔다.

8.2 커밋 메시지

문서 패치와 달리 버그 수정은 리뷰어가 확인할 것이 많다. 그래서 증상, 재현 방법, 원인, 조치 순으로 썼고, 호출 스택은 원인을 보여 주는 다섯 줄만 인용했다. 경고가 왜 두 개인지도 한 줄로 미리 답해 두었다.

태그는 두 가지를 판단했다. Fixes:는 하나만 달았다. 나중 커밋을 되돌려도 경고는 하나로 줄 뿐 사라지지 않으니, 원인은 work가 처음 들어온 커밋이다. Cc: stable은 달지 않았다. 이 work는 큐에 들어간 적이 없

어 취소할 대상 자체가 없고, 경고를 찍은 뒤 함수는 그냥 돌아온다. 메모리 손상이나 use-after-free는 일어나지 않고 실제 피해는 경고와 커널 오염 표시뿐이다. stable 규칙의 “사람들을 괴롭히는 실제 버그”라는 기준에 대면 약하다고 봤다.

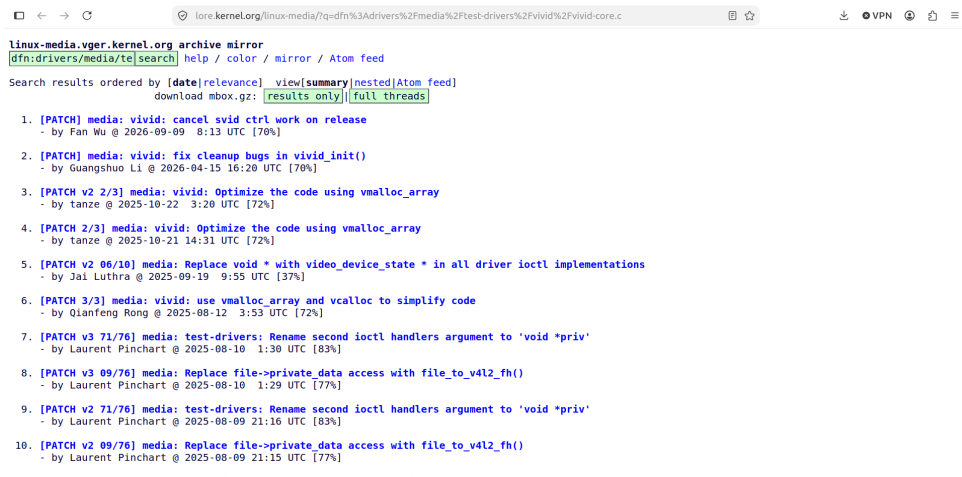
8.3 받는 사람이 하나 늘었다

```
$ ./scripts/get_maintainer.pl 0001-*.patch
Hans Verkuil <...> (maintainer:VIVID..., blamed_fixes:1/1=100%)
Mauro Carvalho Chehab <...> (maintainer:MEDIA INPUT INFRASTRUCTURE)
Dorcas Anono Litunya <...> (blamed_fixes:1/1=100%)
linux-media@vger.kernel.org / linux-kernel@vger.kernel.org
```

앞의 두 패치에는 없던 blamed_fixes 항목이 나왔다. Fixes: 태그를 보고 그 커밋의 작성자를 찾아 준 것이다. 자기 코드에서 나온 문제를 본인이 확인할 수 있도록, 관행대로 참조에 넣었다.

8.4 발송

보내기 직전에 트리가 그사이 바뀌지 않았는지 다시 받아 보고, 목록도 한 번 더 확인했다.



```
linux-media.vger.kernel.org archive mirror
dfn:drivers/media/te|search help / color / mirror / Atom feed
Search results ordered by [date|relevance] view[summary|nested|Atom feed]
download mbox.gz: results only || full threads

1. [PATCH] media: vivid: cancel svid ctrl work on release
   - by Fan Wu @ 2026-09-09 8:13 UTC [70%]

2. [PATCH] media: vivid: fix cleanup bugs in vivid_init()
   - by Guangshuo Li @ 2026-04-15 16:20 UTC [70%]

3. [PATCH v2 2/3] media: vivid: Optimize the code using vmalloc_array
   - by tanze @ 2025-10-22 3:20 UTC [72%]

4. [PATCH 2/3] media: vivid: Optimize the code using vmalloc_array
   - by tanze @ 2025-10-21 14:31 UTC [72%]

5. [PATCH v2 06/10] media: Replace void * with video_device_state * in all driver ioctl implementations
   - by Jai Luthra @ 2025-09-19 9:55 UTC [37%]

6. [PATCH 3/3] media: vivid: use vmalloc_array and vcalloc to simplify code
   - by Qianfeng Rong @ 2025-08-12 3:53 UTC [72%]

7. [PATCH v3 71/76] media: test-drivers: Rename second ioctl handlers argument to 'void *priv'
   - by Laurent Pinchart @ 2025-08-10 1:30 UTC [83%]

8. [PATCH v3 09/76] media: Replace file->private_data access with file_to_v4l2_fh()
   - by Laurent Pinchart @ 2025-08-10 1:29 UTC [77%]

9. [PATCH v2 71/76] media: test-drivers: Rename second ioctl handlers argument to 'void *priv'
   - by Laurent Pinchart @ 2025-08-09 21:16 UTC [83%]

10. [PATCH v2 09/76] media: Replace file->private_data access with file_to_v4l2_fh()
    - by Laurent Pinchart @ 2025-08-09 21:15 UTC [77%]
```

발송 직전의 재확인. 맨 위가 여전히 9월 9일 패치였다.

checkpatch는 오류, 경고, 확인 사항이 모두 0이었다. 커밋 메시지에 넣은 호출 스택 인용도 문제 삼지 않았다. 리허설로 받는 사람을 확인한 뒤 보냈고, 마지막 줄에 Result: 250이 찍혔다.

한 장 요약

실행해서 찾은 버그를 패치로 만들 때 점검하는 체크리스트

발견

- ☐ 경고나 오류를 재현하는 명령을 한 줄로 만들었다
- ☐ 같은 조건에서 두 번 이상 재현되는 것을 확인했다
- ☐ 호출 스택에서 원인 함수와 그 위 호출자를 짚었다

원인

- ☐ 경고를 낸 조건이 소스의 어느 줄인지 찾았다
- ☐ 문제의 두 시점(무엇이 먼저, 무엇이 나중)을 줄 번호로 확인했다
- ☐ 같은 경로의 다른 정리 코드도 안전한지 하나씩 확인해 범위를 좁혔다
- ☐ `git log -S`로 언제 들어온 코드인지 찾아 원인 커밋을 정했다

중복

- ☐ 파일 이름, 증상, 함수 이름으로 각각 검색했다
- ☐ 선례를 읽고 그 영역에서 통하는 방식과 태그 관행을 봤다

수정과 검증

- ☐ 지켜야 할 규칙을 한 문장으로 적고, 그에 맞는 위치를 골랐다
- ☐ 모듈 파일이 소스보다 새것인지 확인했다
- ☐ 가상 머신 커널과 같은 방법으로 빌드했다
- ☐ 고치기 전 숫자와 고친 뒤 숫자를 같은 명령으로 비교했다
- ☐ 원래 동작이 그대로인 것도 확인했다

발송

- ☐ Fixes: 대상을 “되돌리면 사라지는가”로 정했다
- ☐ Cc: stable 여부를 규칙 문서와 대 보고 판단했다
- ☐ `blamed_fixes`로 나온 사람을 참조에 넣었다
- ☐ 직전에 트리와 목록을 다시 확인하고 리허설을 했다